Gravefin: GraphQL Architecture

Complete Technical Specification & Implementation Guide

Version: 1.0.0 (Production) | Status: Verified & Tested (100% Pass) | Engine: GraphQL Yoga v5

1. What GraphQL Is Doing in Gravefin

Gravefin is a blind, trustless, escrow-backed marketplace designed to resurrect abandoned student and developer side projects. In this domain, data entities do not exist in isolation—they form an intricate relational graph connecting dead repositories, anonymous Diggers, forensic autopsies, active escrow commitments, and append-only ledger entries.

- **Graph-Based Single-Round-Trip Retrieval:** Eliminates REST waterfall latency. Clients fetch projects, owners, autopsies, and commitments in a single HTTP POST request.
- **Atomic Escrow State Mutations:** Executes transactional staking and sign-offs directly with Prisma, ensuring ledger synchronization.
- **Automated Privacy & Identity Shielding:** Resolvers dynamically hide emails and credentials, ensuring complete anonymity across all counter-parties.
- **Deterministic Cron Timeouts:** Allows clients and simulation engines to trigger 14-day taker dormancy and 7-day owner ghost resolution.

2. System Architecture & Implementation Layers

The GraphQL architecture is organized into four clean, production-grade layers:

```
+-------------------------------------------------------------------------+
|  1. SCHEMA DEFINITION: lib/graphql/typeDefs.js                          |
|     - Strict GraphQL SDL (Types: Project, User, Commitment, LedgerEntry) |
|     - Defines Queries (projects, leaderboard, ledgerEntries) & Mutations  |
+-------------------------------------------------------------------------+
|  2. RESOLVERS & BUSINESS LOGIC: lib/graphql/resolvers.js                |
|     - Top-level query resolvers with multi-attribute filtering          |
|     - Nested field resolvers: Project.owner, Project.commitments        |
|     - Transactional mutations: commitToProject, acceptDelivery          |
+-------------------------------------------------------------------------+
|  3. SERVERLESS ENDPOINT: pages/api/graphql.js                           |
|     - Powered by GraphQL Yoga v5 with embedded GraphiQL Playground IDE   |
|     - Injects session auth context via parseSession(req)                |
+-------------------------------------------------------------------------+
|  4. IN-APP PROTOCOL EXPLORER: pages/graphql.jsx                         |
|     - Integrated parchment UI page linked from primary navigation bar    |
|     - 4 instant preset queries with live execution timer and JSON viewer |
+-------------------------------------------------------------------------+
```

3. Core Entity Types & Relational Graph

The schema defines complete representations of the Gravefin protocol:

- **Project:** Dead codebase with title, stakeRequired, techTags, revivalScore, autopsy, and owner.
- **Commitment:** Active excavation sprint with stakeLocked, ghostDeadlineAt, submissionNotes, milestones.
- **User:** Pseudonymous Digger with alias, credits, reputation, ghostStrikes, and audit history.
- **LedgerEntry:** Immutable financial transaction with type (STAKE_LOCK, STAKE_REFUND), amount, and notes.

4. Real Working Query & Mutation Examples

A. Querying Catalog Specimens with Nested Relational Autopsies:

```
query GetCatalogSpecimens {
  projects(limit: 2) {
    id
    title
    status
    stakeRequired
    techTags
    revivalScore
    revivalDetails { overallScore, effortEstimate }
    autopsy { cause, difficulty }
    owner { alias, reputation }
  }
}
```

B. Staking Escrow Collateral & Claiming Codebase (Mutation):

```
mutation ClaimSpecimen {
  commitToProject(projectId: "p1") {
    id
    status
    stakeLocked
    committedAt
    taker { alias, credits }
  }
}
```

C. Autonomous Timeout Cron Trigger (Mutation):

```
mutation ExecuteCronSweep {
  triggerTimeoutCheck(secret: "graveyard_secret_key", simulateDaysElapsed: 7) {
    success
    effectiveDate
    events
    summary
  }
}
```

5. How to Access, Test & Verify

- **In-App UI Explorer:** Open http://localhost:3000/graphql or click GraphQL in the top header.
- **Interactive GraphiQL IDE:** Open http://localhost:3000/api/graphql in any browser for auto-complete & docs.
- **Automated Test Suite:**Run `npm run test:graphql` to verify query compilation and mutations.